Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected Program: Q5 – Prime Number Checking

A. Diagnose the Bottleneck

The original program checks every divisor from 2 up to the square root of the given number. This performs unnecessary operations because even numbers greater than 2 cannot be prime.

B. Define the Design Objective

The objective is to reduce the number of division operations by checking 2 separately and then checking only odd divisors.

```
#include <stdio.h>


int main()
{
   unsigned long numbers[] = {97, 100, 7919, 1000003};
   unsigned long n, i;
   int prime, divisions;


   for (int j = 0; j < 4; j++)
   {
      n = numbers[j];
      prime = 1;
      divisions = 0;


      if (n < 2)
         prime = 0;


      else if (n == 2)
         prime = 1;


      else if (n % 2 == 0)
```

```
        {
            divisions++;
            prime = 0;
        }
        else
        {
            for (i = 3; i <= n / i; i += 2)
            {
                divisions++;

                if (n % i == 0)
                {
                    prime = 0;
                    break;
                }
            }
        }

        printf("%lu: %s, Divisions = %d\n",
               n, prime ? "Prime" : "Not Prime", divisions);
    }

    return 0;
}
```

D:Predict Before Implementing

The improved program is expected to reduce division operations by skipping even divisors greater than 2 while producing the same prime-number results.

Implement and Validate

Output:

97: Prime, Divisions = 4

100: Not Prime, Divisions = 1

7919: Prime, Divisions = 44

1000003: Prime, Divisions = 500

Comparison Table:

Number	Original	Improved
97	8	4
100	1	1
7919	87	44
1000003	999	500

F: Defend the Final Design

The improved program is more efficient because it skips unnecessary even divisors. It reduces the number of division operations and improves execution efficiency. Both programs produce correct prime-number results. Therefore, the redesigned program is better for checking large numbers.